Week 3 – Feature Engineering on the Updated Retail Dataset

Project

Demand Forecasting for Retail Inventory Optimization

Objective

In this section, we explore four text vectorization methods:

1. **Bag of Words (BoW)**
2. **TF-IDF**
3. **Word2Vec**
4. **BERT / Transformer Embeddings**

These methods convert text-like fields into numerical vectors that can later be combined with structured retail features for machine learning models.

Important Note

The updated retail dataset is primarily a **structured tabular dataset**.
The text-like fields in this dataset are mostly **short categorical/business labels** such as:

- `Category`
- `Store_Type`
- `Festival_Name`
- `Festival_Type`
- `Product_Name`
- `Sub_Category`
- `Brand_Name`
- `Brand_Tier`
- `Promotion_Type`

Therefore, these vectorization methods are being explored as part of the project requirement, but they are **not the primary feature engineering strategy** for this dataset.
Later, we will also apply more appropriate tabular feature engineering methods.

Loading [MathJax]/extensions/Safe.js

Target Variable

The target variable remains:

`Daily_Units_Sold`

In [1]:
```
import numpy as np
import pandas as pd
import scipy
import sklearn
import gensim

print("numpy:", np.__version__)
print("pandas:", pd.__version__)
print("scipy:", scipy.__version__)
print("sklearn:", sklearn.__version__)
print("gensim:", gensim.__version__)
```

```
numpy: 1.26.4
pandas: 2.2.2
scipy: 1.13.1
sklearn: 1.6.1
gensim: 4.3.3
```

In [2]:
```
# @title Import basic libraries for data handling
#import pandas as pd
#import numpy as np

# Import libraries for text processing
import re
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

# Import libraries for Word2Vec
from gensim.models import Word2Vec

# Import library for contextual embeddings
!pip -q install sentence-transformers

from sentence_transformers import SentenceTransformer

# Display options for readability
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 120)
```

```
/usr/local/lib/python3.12/dist-packages/sentence_transformers/cross_encoder/
CrossEncoder.py:11: TqdmExperimentalWarning: Using `tqdm.autonotebook.tqdm`
in notebook mode. Use `tqdm.tqdm` instead to force console mode (e.g. in jup
yter console)
  from tqdm.autonotebook import tqdm, trange
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without
Triton certain kernels will not work
```

Load the Updated Dataset

Loading [MathJax]/extensions/Safe.js

We first load the updated retail dataset and inspect its shape and column names.

The updated dataset is expected to contain:

- 2,000 rows
- 58 columns

In [3]:
```
# @title Load the updated dataset
df = pd.read_csv("/content/master_retail_dataset_v2.csv")

# Display shape of the dataset
print("Dataset shape:", df.shape)

# Preview first 5 rows
df.head()
```

```
Dataset shape: (2000, 58)
```

Out[3]:

	Date	DayOfWeek	Month	Year	Is_Weekend	Is_Holiday	Product_ID	Cate
0	04/13/2023	3	4	2023	0	0	1002	Gro
1	12/15/2023	4	12	2023	0	0	1014	H
2	09/28/2023	3	9	2023	0	0	1015	Electro
3	04/17/2023	0	4	2023	0	0	1012	Gro
4	03/13/2023	0	3	2023	0	0	1016	Gro

Identify Text-Like Features

The dataset is mostly structured, but some fields contain business labels that can be treated as text-like features for vectorization.

The selected columns are:

- `Category`
- `Store_Type`
- `Festival_Name`
- `Festival_Type`
- `Product_Name`
- `Sub_Category`
- `Brand_Name`
- `Brand_Tier`

Loading [MathJax]/extensions/Safe.js

- `Promotion_Type`

In [4]:
```
# @title List of text-like columns selected for vectorization
text_columns = [
    "Category",
    "Store_Type",
    "Festival_Name",
    "Festival_Type",
    "Product_Name",
    "Sub_Category",
    "Brand_Name",
    "Brand_Tier",
    "Promotion_Type"
]

# Show number of unique values in each selected text column
for col in text_columns:
    print(f"{col}: {df[col].nunique()} unique values")
```

```
Category: 5 unique values
Store_Type: 2 unique values
Festival_Name: 8 unique values
Festival_Type: 3 unique values
Product_Name: 100 unique values
Sub_Category: 20 unique values
Brand_Name: 19 unique values
Brand_Tier: 3 unique values
Promotion_Type: 4 unique values
```

Create a Combined Text Field

Most vectorization methods work on a single text document per row.

To apply these methods, we combine the selected text-like columns into one consolidated text field called:

`combined_text`

This creates a simple text representation of each retail record.

In [5]:
```
# Fill missing values with empty string just in case
df[text_columns] = df[text_columns].fillna("")

# Combine selected columns into one text string per row
df["combined_text"] = df[text_columns].astype(str).agg(" ".join, axis=1)

# Preview the combined text
df[["combined_text"]].head()
```

Loading [MathJax]/extensions/Safe.js

Out[5]:

	combined_text
0	Grocery Urban Nestle Beverages 1002 Beverages Nestle Premium
1	Home Rural Tupperware Decor 1014 Decor Tupperware Budget Flash Sale
2	Electronics Urban Sony Computers 1015 Computers Sony Mid
3	Grocery Urban Nestle Spices 1012 Spices Nestle Premium Flash Sale
4	Grocery Urban Tata Spices 1016 Spices Tata Premium

Basic Text Cleaning

Before vectorization, we perform light text cleaning:

- convert text to lowercase
- remove extra spaces
- keep only letters, numbers, and spaces

This is a simple preprocessing step suitable for the current dataset.

In [6]:

```
# Function to clean text
def clean_text(text):
    """
    Clean input text by:
    1. converting to lowercase
    2. removing special characters
    3. removing extra spaces
    """
    text = text.lower()
    text = re.sub(r"[^a-zA-Z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

# Apply cleaning
df["clean_text"] = df["combined_text"].apply(clean_text)

# Preview cleaned text
df[["combined_text", "clean_text"]].head()
```

Loading [MathJax]/extensions/Safe.js

Out[6]:

	combined_text	clean_text
0	Grocery Urban Nestle Beverages 1002 Beverages Nestle Premium	grocery urban nestle beverages 1002 beverages nestle premium
1	Home Rural Tupperware Decor 1014 Decor Tupperware Budget Flash Sale	home rural tupperware decor 1014 decor tupperware budget flash sale
2	Electronics Urban Sony Computers 1015 Computers Sony Mid	electronics urban sony computers 1015 computers sony mid
3	Grocery Urban Nestle Spices 1012 Spices Nestle Premium Flash Sale	grocery urban nestle spices 1012 spices nestle premium flash sale
4	Grocery Urban Tata Spices 1016 Spices Tata Premium	grocery urban tata spices 1016 spices tata premium

1. Bag of Words (BoW)

Concept

Bag of Words converts each document into a vector of **word counts**.

If a word appears more often in a row's text, its value in the vector is higher.

Formula

For document d_i and term w_j:

$ x_{ij} = \text{count}(w_j, d_i) $

where:

- x_{ij} = count of word w_j in document d_i

Why We Are Using It

BoW is:

- simple
- easy to interpret
- useful as a baseline vectorization method

Limitation

It does **not** capture:

Loading [MathJax]/extensions/Safe.js

- semantic meaning
- word order
- context

In [7]:
```
# Create a CountVectorizer object
# max_features limits the vocabulary size so the matrix remains manageable
bow_vectorizer = CountVectorizer(max_features=100)

# Learn vocabulary and transform text into count vectors
X_bow = bow_vectorizer.fit_transform(df["clean_text"])

# Convert sparse matrix to DataFrame for inspection
bow_df = pd.DataFrame(
    X_bow.toarray(),
    columns=bow_vectorizer.get_feature_names_out()
)

print("BoW matrix shape:", bow_df.shape)
bow_df.head()
```

```
BoW matrix shape: (2000, 95)
```

Out[7]:

	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011	1012
0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0	1
4	0	0	0	0	0	0	0	0	0	0	0	0	0

Observation

Bag of Words produced a **2000 × 95** sparse feature matrix. Although the vectorizer was configured with `max_features=100`, the effective vocabulary extracted from the cleaned combined text was **95 tokens**.

The generated vocabulary is dominated by:

- short business labels such as categories, brand tiers, and promotion words
- product/store identifiers such as `1002`, `1014`, and similar numeric tokens
- repeated business terms such as product or sub-category names

This confirms that BoW is mainly capturing **token presence and repetition** in structured label-style text rather than rich natural language context. For this dataset, it works as a simple baseline representation, but it mostly reflects category, brand, promotion, and identifier information.

Loading [MathJax]/extensions/Safe.js

Note on Combined Text Representation

In the Bag of Words approach, all selected text-like columns were combined into a single document per row.

This is a standard NLP baseline approach, but it may mix semantically different fields such as:

- product
- brand
- store type
- promotion
- festival

This limitation is examined further in the next section.

2. TF-IDF

Concept

TF-IDF stands for **Term Frequency – Inverse Document Frequency**.

It improves upon Bag of Words by giving:

- higher weight to words that are important in a specific document
- lower weight to very common words across many documents

Formula

$ \text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) $

where:

$ \text{IDF}(t) = \log\left(\frac{N}{df(t)}\right) $

- N = total number of documents
- $df(t)$ = number of documents containing term t

Why We Are Using It

TF-IDF is useful because it highlights **informative terms** and reduces the impact of overly common words.

Loading [MathJax]/extensions/Safe.js

Limitation

It still does not capture semantic meaning or context.

In [8]:
```
# Create TF-IDF vectorizer
tfidf_vectorizer = TfidfVectorizer(max_features=100)

# Fit and transform the cleaned text
X_tfidf = tfidf_vectorizer.fit_transform(df["clean_text"])

# Convert sparse matrix to DataFrame for inspection
tfidf_df = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out()
)

print("TF-IDF matrix shape:", tfidf_df.shape)
tfidf_df.head()
```

```
TF-IDF matrix shape: (2000, 95)
```

Out[8]:

	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011
0	0.0	0.0	0.327848	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Observation

TF-IDF produced a **2000 × 95** weighted feature matrix, which matches the effective vocabulary size observed in the Bag of Words representation.

Compared with BoW, TF-IDF assigns higher weight to terms that are more specific to individual records and lower weight to terms that appear very frequently across many rows. In this dataset, the weighting is especially useful because the combined text contains repeated business labels such as category, brand, and promotion terms.

However, the output also shows that many important tokens are still short structured labels or numeric identifiers rather than descriptive sentences. Therefore, TF-IDF is more informative than BoW for highlighting distinguishing labels, but it is still limited by the short, categorical nature of the text fields.

Limitation of Combined Text TF-IDF

Loading [MathJax]/extensions/Safe.js

The previous TF-IDF approach treated all text-like columns as one combined document per row.

However, this dataset contains multiple text-like fields with different meanings, such as:

- `Category`
- `Store_Type`
- `Festival_Name`
- `Festival_Type`
- `Product_Name`
- `Sub_Category`
- `Brand_Name`
- `Brand_Tier`
- `Promotion_Type`

Combining them into a single text string mixes unrelated semantic categories. As a result, the TF-IDF representation may lose interpretability because words from different business domains are treated as if they belong to the same context.

Improved Approach: Column-wise TF-IDF Vectorization

To preserve the semantic meaning of each text-like feature, TF-IDF can be applied separately to each column rather than combining all columns into one document.

This approach helps to:

- preserve feature-specific context
- reduce semantic mixing
- improve interpretability

In this method, each text column is vectorized independently, and the resulting feature matrices are then concatenated.

In [9]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

# Create an empty list to store TF-IDF DataFrames for each column
tfidf_features = []

# Apply TF-IDF separately to each text column
for col in text_columns:
    # Initialize TF-IDF vectorizer
    # max_features is limited to keep the feature space manageable
    vectorizer = TfidfVectorizer(max_features=20)

    # Convert the current column to string
    data = df[col].astype(str)
```

Loading [MathJax]/extensions/Safe.js

```
    # Fit and transform the column
    col_matrix = vectorizer.fit_transform(text_data)

    # Convert sparse matrix to DataFrame
    col_df = pd.DataFrame(
        col_matrix.toarray(),
        columns=[f"{col}_{word}" for word in vectorizer.get_feature_names_ou
    )

    # Store the result
    tfidf_features.append(col_df)

# Combine all column-wise TF-IDF features into one DataFrame
tfidf_combined_df = pd.concat(tfidf_features, axis=1)

# Display shape and preview
print("Column-wise TF-IDF shape:", tfidf_combined_df.shape)
tfidf_combined_df.head()
```

```
Column-wise TF-IDF shape: (2000, 94)
```

Out[9]:

	Category_beauty	Category_clothing	Category_electronics	Category_grocery	Cate
0	0.0	0.0	0.0	1.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	1.0	0.0	
3	0.0	0.0	0.0	1.0	
4	0.0	0.0	0.0	1.0	

Observation

The column-wise TF-IDF approach preserves the semantic role of each feature by vectorizing each column independently.

Compared to the combined-text TF-IDF approach:

- semantic mixing is reduced
- feature meaning is preserved more clearly
- interpretability is improved

This makes the representation more suitable for structured datasets in which different text columns represent different business concepts.

However, these fields are still short categorical labels rather than natural language text. Therefore, although column-wise TF-IDF is an improvement over combined-text TF-IDF, traditional categorical encoding methods such as one-hot encoding and frequency encoding remain more appropriate for this dataset.

Loading [MathJax]/extensions/Safe.js

Prepare Tokens for Word2Vec

Word2Vec requires tokenized text input.

So we convert each cleaned text row into a list of words.

In [10]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

# Create an empty list to store TF-IDF DataFrames for each column
tfidf_features = []

# Apply TF-IDF separately to each text column
for col in text_columns:
    # Initialize TF-IDF vectorizer
    # max_features is limited to keep the feature space manageable
    vectorizer = TfidfVectorizer(max_features=20)

    # Convert the current column to string
    text_data = df[col].astype(str)

    # Fit and transform the column
    col_matrix = vectorizer.fit_transform(text_data)

    # Convert sparse matrix to DataFrame
    col_df = pd.DataFrame(
        col_matrix.toarray(),
        columns=[f"{col}_{word}" for word in vectorizer.get_feature_names_ou
    )

    # Store the result
    tfidf_features.append(col_df)

# Combine all column-wise TF-IDF features into one DataFrame
tfidf_combined_df = pd.concat(tfidf_features, axis=1)

# Display shape and preview
print("Column-wise TF-IDF shape:", tfidf_combined_df.shape)
tfidf_combined_df.head()
```

```
Column-wise TF-IDF shape: (2000, 94)
```

Out[10]:

	Category_beauty	Category_clothing	Category_electronics	Category_grocery	Cate
0	0.0	0.0	0.0	1.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	1.0	0.0	
3	0.0	0.0	0.0	1.0	
4	0.0	0.0	0.0	1.0	

Loading [MathJax]/extensions/Safe.js

In [12]:
```
# Tokenize cleaned text by simple whitespace splitting
df["tokens"] = df["clean_text"].apply(lambda x: x.split())

# Preview tokenized text
df[["clean_text", "tokens"]].head()
```

Out[12]:

	clean_text	tokens
0	grocery urban nestle beverages 1002 beverages nestle premium	[grocery, urban, nestle, beverages, 1002, beverages, nestle, premium]
1	home rural tupperware decor 1014 decor tupperware budget flash sale	[home, rural, tupperware, decor, 1014, decor, tupperware, budget, flash, sale]
2	electronics urban sony computers 1015 computers sony mid	[electronics, urban, sony, computers, 1015, computers, sony, mid]
3	grocery urban nestle spices 1012 spices nestle premium flash sale	[grocery, urban, nestle, spices, 1012, spices, nestle, premium, flash, sale]
4	grocery urban tata spices 1016 spices tata premium	[grocery, urban, tata, spices, 1016, spices, tata, premium]

3. Word2Vec

Concept

Word2Vec converts words into **dense numerical embeddings**.

Unlike BoW and TF-IDF, Word2Vec can capture some degree of **semantic similarity** between words.

It learns word representations by examining surrounding words in context.

Two common architectures are:

- **CBOW**: predicts a word from surrounding words
- **Skip-Gram**: predicts surrounding words from a given word

Why We Are Using It

Word2Vec provides:

- dense low-dimensional vectors
- better semantic representation than frequency-based methods

Limitation

Loading [MathJax]/extensions/Safe.js

Word2Vec is **context-independent**, meaning a word gets the same embedding everywhere.

For this dataset, Word2Vec is exploratory because the text fields are short business labels rather than full sentences.

In [13]:
```
# Train a simple Word2Vec model on tokenized text
# vector_size = embedding dimension
# window = context window size
# min_count = minimum frequency required for a word to be learned
# workers = number of CPU cores used
w2v_model = Word2Vec(
    sentences=df["tokens"],
    vector_size=50,
    window=3,
    min_count=1,
    workers=4,
    sg=1  # sg=1 means Skip-Gram
)

print("Word2Vec vocabulary size:", len(w2v_model.wv.index_to_key))
print("Sample words:", w2v_model.wv.index_to_key[:10])
```

```
Word2Vec vocabulary size: 96
Sample words: ['urban', 'rural', 'premium', 'mid', 'budget', 'grocery', 'bea
uty', 'home', 'electronics', 'computers']
```

Create Document-Level Word2Vec Vectors

Word2Vec learns vectors for individual words, but our machine learning model later needs **one vector per row**.

So we compute the **average of all word vectors in each row**.

Formula

For a document with n words:

$ \text{Document Vector} = \frac{1}{n}\sum_{i=1}^{n} \vec{w_i} $

where:

- $\vec{w_i}$ = embedding of word i

In [14]:
```
# Function to average word vectors for one document
def average_word2vec(tokens, model, vector_size):
    """
    Return the average Word2Vec embedding for a list of tokens.
    If no token is found, return a zero vector.
    """
    vectors = [model.wv[word] for word in tokens if word in model.wv]
```

Loading [MathJax]/extensions/Safe.js

```
    if len(valid_vectors) == 0:
        return np.zeros(vector_size)

    return np.mean(valid_vectors, axis=0)

# Apply function row by row
w2v_vectors = np.array([
    average_word2vec(tokens, w2v_model, 50)
    for tokens in df["tokens"]
])

# Convert to DataFrame
w2v_df = pd.DataFrame(w2v_vectors, columns=[f"w2v_{i}" for i in range(50)])

print("Word2Vec document matrix shape:", w2v_df.shape)
w2v_df.head()
```

```
Word2Vec document matrix shape: (2000, 50)
```

Out[14]:

	w2v_0	w2v_1	w2v_2	w2v_3	w2v_4	w2v_5	w2v_6	w2v
0	-0.092853	-0.078781	0.060864	0.057939	-0.145222	-0.038916	0.234718	0.1174
1	-0.049718	0.114319	0.163936	0.089554	-0.152529	0.139797	0.194684	0.5267
2	0.009395	0.186711	-0.116326	0.156620	0.175861	-0.239864	0.126047	0.1361
3	-0.068717	-0.054978	0.119850	0.099447	-0.184870	-0.033369	0.247362	0.1151
4	-0.112688	-0.091370	0.139225	0.101014	-0.277112	-0.042811	0.325036	0.0183

Observation

Word2Vec learned a vocabulary of **96 unique tokens** and produced a **2000 × 50** dense document embedding matrix after averaging word vectors row by row.

The sample vocabulary includes meaningful business words such as:

- `urban`, `rural`
- `premium`, `mid`, `budget`
- `grocery`, `beauty`, `home`, `electronics`
- `computers`

This indicates that Word2Vec is capturing semantic relationships among store, product, and brand-related terms rather than only raw counts. However, because each row contains only short label-style text instead of full natural-language sentences, the learned semantic context is limited. As a result, Word2Vec provides a compact dense representation, but its benefit in this dataset is still exploratory rather than strongly language-driven.

Loading [MathJax]/extensions/Safe.js

4. BERT / Transformer Embeddings

Concept

BERT-based embeddings produce **context-aware vector representations**.

Unlike Word2Vec, BERT can generate embeddings that reflect the context in which words appear.

For this notebook, we use a lightweight sentence-transformer model to generate one embedding per row.

Why We Are Using It

Transformer embeddings provide:

- richer semantic representation
- context-aware text encoding
- strong performance in modern NLP tasks

Limitation

They are more computationally expensive than BoW, TF-IDF, or Word2Vec.

In this dataset, BERT is exploratory because the text fields are short labels and not long natural-language sentences.

In [15]:
```
# Load a lightweight sentence transformer model
bert_model = SentenceTransformer("all-MiniLM-L6-v2")

# Generate embeddings for each text row
bert_vectors = bert_model.encode(
    df["clean_text"].tolist(),
    show_progress_bar=True
)

# Convert embeddings to DataFrame
bert_df = pd.DataFrame(
    bert_vectors,
    columns=[f"bert_{i}" for i in range(bert_vectors.shape[1])]
)

print("BERT embedding matrix shape:", bert_df.shape)
bert_df.head()
```

Loading [MathJax]/extensions/Safe.js

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: U
serWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings t
ab (https://huggingface.co/settings/tokens), set it as secret in your Google
Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
public models or datasets.
  warnings.warn(
BERT embedding matrix shape: (2000, 384)
```

Out[15]:

	bert_0	bert_1	bert_2	bert_3	bert_4	bert_5	bert_6	b
0	0.019519	0.035894	0.006501	0.005474	0.032532	0.009000	-0.032032	0.0
1	-0.027409	0.005514	-0.001803	-0.005229	0.057841	-0.036739	-0.042057	-0.0
2	0.003602	0.013918	0.018764	-0.133172	0.025483	0.002544	-0.014594	0.0
3	-0.064240	0.009937	0.008962	0.042035	0.023387	0.032920	-0.056224	0.0
4	-0.036815	-0.035842	-0.049133	0.040742	-0.019654	0.049562	-0.030679	-0.0

Observation

BERT produced a **2000 × 384** embedding matrix, which is the richest and highest-dimensional representation among the four methods tested.

This method generates contextual sentence-level embeddings, so each row is represented by a dense vector rather than a sparse count table. In theory, this is more expressive than BoW, TF-IDF, or Word2Vec.

However, in this dataset the input text is still composed mostly of short business labels such as category names, brand names, product labels, promotion types, and identifiers. Because the rows are not long natural-language descriptions, BERT's contextual advantage is present but likely underutilized. Therefore, BERT provides the most advanced representation technically, but its practical gain for this structured retail dataset may be limited.

Compare the Four Vectorization Outputs

This section compares the dimensionality of the four vectorization methods.

Loading [MathJax]/extensions/Safe.js

In [16]:
```
# Compare the shape of all vector outputs
comparison_df = pd.DataFrame({
    "Method": ["Bag of Words", "TF-IDF", "Word2Vec", "BERT"],
    "Rows": [
        bow_df.shape[0],
        tfidf_df.shape[0],
        w2v_df.shape[0],
        bert_df.shape[0]
    ],
    "Columns / Features": [
        bow_df.shape[1],
        tfidf_df.shape[1],
        w2v_df.shape[1],
        bert_df.shape[1]
    ]
})

comparison_df
```

Out[16]:

	Method	Rows	Columns / Features
0	Bag of Words	2000	95
1	TF-IDF	2000	95
2	Word2Vec	2000	50
3	BERT	2000	384

Summary and Interpretation

The four vectorization methods were implemented successfully on the text-like fields of the updated retail dataset.

Summary of Methods

- **Bag of Words (BoW)**
  Produced a **2000 × 95** sparse matrix based on token counts.
- **TF-IDF**
  Produced a **2000 × 95** weighted matrix that emphasizes more distinctive terms.
- **Word2Vec**
  Learned a **96-token vocabulary** and generated a **2000 × 50** dense embedding matrix.
- **BERT / Transformer Embeddings**
  Produced a **2000 × 384** contextual embedding matrix.

Loading [MathJax]/extensions/Safe.js

Overall Interpretation

The results confirm that these methods can be applied successfully to the selected text-like fields. However, the underlying text in this dataset is mostly composed of:

- short categorical labels
- brand and product names
- promotion labels
- numeric identifiers

rather than long natural-language text.

As a result:

- **BoW** and **TF-IDF** mainly capture token presence and relative importance
- **Word2Vec** captures limited semantic similarity among business terms
- **BERT** provides the richest representation, but its contextual power is not fully utilized in short label-based records

Practical Conclusion

These vectorization methods satisfy the project requirement and are useful for comparative analysis. However, for this retail dataset, they should be treated as **secondary / exploratory text representations** rather than the primary feature engineering strategy.

More suitable methods for the full forecasting pipeline are likely to be:

- categorical encoding
- date and calendar feature extraction
- lag features
- rolling statistics
- price-promotion interaction features
- store/product aggregation features

Optional Step: Save Vectorized Outputs

If needed, the generated feature matrices can be saved for use in later machine learning experiments.

In [17]:
```
# Save vectorized outputs as CSV files for later use
bow_df.to_csv("bow_features.csv", index=False)
tfidf_df.to_csv("tfidf_features.csv", index=False)
w2v_df.to_csv("word2vec_features.csv", index=False)
bert_df.to_csv("bert_features.csv", index=False)
```

Loading [MathJax]/extensions/Safe.js

```
print("Feature files saved successfully.")
```

```
Feature files saved successfully.
```

Final Conclusion

The four vectorization methods were applied successfully to the text-like fields of the updated retail dataset.

From the actual outputs:

- **BoW** and **TF-IDF** each produced **95 features**
- **Word2Vec** produced **50 dense features**
- **BERT** produced **384 dense contextual features**

These results show that the text-like fields can be transformed into machine-readable vectors. However, since the dataset is primarily structured and categorical rather than text-heavy, these methods are best viewed as exploratory approaches for this project.

For the broader retail demand forecasting task, more suitable feature engineering methods will include categorical encoding, calendar-based features, lag variables, rolling statistics, interaction variables, and aggregation-based features.

Summary and Interpretation

The text-like fields of the updated retail dataset were explored using four vectorization methods:

- Bag of Words (BoW)
- TF-IDF
- Word2Vec
- BERT / Transformer Embeddings

In addition, an improved **column-wise TF-IDF** approach was tested to preserve the semantic meaning of individual text-like columns.

Key Findings

- BoW and TF-IDF on combined text provide baseline vector representations.
- However, combining all text-like fields into one document mixes different business meanings such as product, brand, promotion, and store type.
- Column-wise TF-IDF improves interpretability by preserving feature-specific context.

Loading [MathJax]/extensions/Safe.js

- Word2Vec and BERT generate dense embeddings, but their value is limited because the dataset does not contain long natural-language text.

Final Interpretation

Although these vectorization methods were successfully applied, the dataset is fundamentally a **structured categorical retail dataset**, not a text-heavy NLP dataset.

Therefore, the most appropriate techniques for this dataset are still:

- one-hot encoding
- frequency encoding
- target encoding
- business-oriented feature engineering

The vectorization methods are useful as exploratory techniques, but they are not the primary feature engineering strategy for this problem.

Comparison: Combined TF-IDF vs Column-wise TF-IDF

The two TF-IDF strategies differ in how they treat the text-like columns.

Approach	Description	Strength	Limitation
Combined TF-IDF	All text columns merged into one document per row	Simple baseline	mixes unrelated semantics
Column-wise TF-IDF	Each text column vectorized separately	preserves feature meaning better	still less suitable than categorical encoding for this dataset

Loading [MathJax]/extensions/Safe.js